

ECSH: Certifying Structural Capability Birth in Self-Modifying Systems via Reoptimized Ancestral Counterfactuals

Ryutaro Yonezu | Independent Researcher, Japan

Version 1.0.0 — 17 September 2026

Paper DOI: 10.5281/zenodo.22806440 | Software/Evidence DOI: 10.5281/zenodo.22806442

Repository: <https://github.com/yonezaemon1-hub/ecsh-structural-capability-birth-certification>

Release status. Public-release candidate revised on 17 September 2026 after submission of a Japanese patent application for the narrow ECSH core on 19 August 2026. This revision refreshes related-work chronology, claim-boundary wording, and reproducibility provenance only; it does not introduce a new ECSH mechanism. The manuscript remains limited to technical material supported by the submitted disclosure and to post-submission verification using the same disclosed synthetic configuration. The earlier candidate-level historical raw artifact was not recovered in a bounded local search; the public evidence is therefore identified as fresh deterministic regeneration from the frozen public source/specification. The patent submission does not establish legal novelty, inventive step, validity, or technical correctness.

Abstract

Self-modifying systems can improve through at least two distinct mechanisms: a child configuration may introduce a structural extension relative to the capability class available before the change, or it may find a better point inside that already-available class. Parent-relative improvement tests do not distinguish these mechanisms. This paper introduces ECSH, a structural-attribution protocol for deciding when a candidate structural element may be treated as a new downstream-generatable capability relative to a declared ancestral class. ECSH preserves the identity of a candidate child C , records or references ancestral capability-state information M_0 describing the pre- c structural vocabulary, generation or mutation rules, permissions, dependencies, or versioned state, and constructs an ancestral capability class A_0 that disables c and required equivalents while preserving legitimate pre- c reoptimization. A_0 is searched, reoptimized, enumerated, or upper-bounded to derive counterfactual comparison information B_0 , while a pre-specified refutation-sufficiency condition Q_0 checks whether the old class received an adequate opportunity to falsify the structural-extension hypothesis. C is compared with B_0 on evaluation data informationally isolated from both child generation and A_0 reoptimization. Operational adoption remains separate from structural attribution, and downstream generatability is updated only when the M_0/A_0 boundary is consistent, Q_0 is satisfied, and C passes a pre-specified non-reducibility rule. In a synthetic control with known candidate classes, canonical seed 20260816 over 1,200 lifetimes produced 97.55% structural-extension recall, 24 false heritable births (0.020 per lifetime), and 99.8976% precision. Three post-submission fresh seeds (20260819–20260821) produced mean recall 97.5014%, mean false births per lifetime 0.01944, and mean precision 99.9004%. On 17 September 2026, all four reported seeds were freshly regenerated from the frozen public reproduction source and specification, yielding 960,000 candidate rows and independently recomputed aggregates matching the manuscript values. On the canonical seed, false births per lifetime were 126.1117 for Child>Parent alone, 77.6092 for a Parent-relative statistical gate, 5.4383 for Shadow comparison without cumulative error control, and 0.020 for the reproduced ECSH configuration. These results calibrate gate behavior in a synthetic setting; they do not establish real-world structural capability birth.

Keywords: self-modification; structural attribution; capability expansion; counterfactual evaluation; ancestral capability state; ancestral capability class; refutation sufficiency; open-ended AI; heredity gate; search-space control

1. Introduction

Open-ended self-improvement systems increasingly modify code, model structure, tools, representations, workflows, and search procedures. Empirical systems such as the Darwin Gödel Machine (DGM) show that recursive code modification can produce large benchmark gains, while the Statistical Gödel Machine (SGM) studies statistically controlled acceptance of recursive modifications. These systems sharpen an important question: when a descendant improves, what exactly has improved? A child can outperform its parent because it contains a genuinely enabling new

structural element, but it can also outperform because it tuned parameters better, reordered existing operations, recombined old modules, exploited a better initialization, or simply searched the old space more effectively.

This distinction matters whenever a system does more than choose a better incumbent. If a candidate element is allowed to become part of the grammar, registry, permission set, tool set, representation vocabulary, or generation policy used to create later descendants, then a mistaken attribution changes the future search space. A one-round evaluation error can therefore become a persistent structural expansion. Conversely, a system that refuses all new structures unless their contribution is attributable risks freezing its representational vocabulary.

ECSH addresses this attribution problem by changing the counterfactual and making its boundary auditable. Instead of asking only whether Child C is better than Parent P, it asks whether C remains better after the pre-c capability class is reconstructed from versioned ancestral capability-state information M_0 and allowed to defend itself. The candidate structural element c is disabled, the old structural vocabulary and generation or mutation rules define A_0 , and A_0 is reoptimized under a pre-specified refutation-sufficiency condition Q_0 . The best attainable performance, a distributional bound, or another valid summary of A_0 becomes counterfactual comparison information B_0 . The structural claim is then tested on evaluation data isolated from both candidate generation and ancestral reoptimization.

The key design choice is that structural attribution controls future generatability rather than operational adoption. A child may be useful enough to retain or deploy while its new element remains unavailable to later candidate generation. In this way, ECSH treats “useful improvement” and “structural capability birth” as separate system decisions.

This paper makes four contributions:

- A precise distinction between Parent-relative improvement and structural non-reducibility relative to a reoptimized pre-c capability class.
- A concrete fail-closed certification protocol that reconstructs a candidate-disabled ancestral capability class from M_0 , checks A_0 -boundary integrity and refutation sufficiency Q_0 , derives counterfactual comparison information, evaluates on informationally isolated data, and conditionally updates downstream generatability.
- An explicit separation between the operational channel and the structural-heredity channel, allowing useful tuning or refactoring to be retained without treating it as a new structural primitive.
- A reproducible synthetic-control calibration showing that Parent-relative gates can produce large numbers of false structural births, while the reproduced ECSH configuration substantially reduces them without eliminating structural-extension recall.

Claim boundary. “Structural birth” in this paper is always relative to a specified ancestral capability-state record M_0 , the resulting ancestral capability class A_0 , equivalence constraints, the declared refutation-sufficiency condition Q_0 , the evaluation-isolation protocol, and the decision rule. ECSH does not claim an absolute proof that no conceivable alternative implementation could match the child.

2. Problem Formulation

2.1 Parent-relative improvement is not structural attribution

Let P be a parent computational configuration and C a child produced by a modification process. Suppose C introduces a candidate structural element c . A conventional acceptance rule may compare scores $S(C)$ and $S(P)$, possibly with statistical uncertainty, and accept C if the improvement is sufficiently convincing. Such a rule answers an operational question: is this child preferable to the current incumbent? It does not answer the attribution question: was c necessary to leave the pre-c capability class?

The distinction is clearest when the old system was under-optimized. If P occupies a poor point in an already expressive search space, a child containing c may outperform P even though a c -free reconfiguration of existing components can match or exceed C . Registering c as a new heritable structural capability in that case conflates search progress with search-space expansion.

2.2 Ancestral capability state and class

Let M_0 denote ancestral capability-state information that identifies the structural vocabulary, generation or mutation rules, permissions, dependencies, type constraints, masks, versions, snapshots, manifests, hashes, commitments, or

equivalent state that determined what was available before c . Define V_0 and R_0 from M_0 . The ancestral capability class $A_0(c; M_0)$ is the set of computational configurations that remain generatable or reoptimizable within that recorded pre- c availability boundary while c is disabled. When a trivial wrapper, rename, equivalent API, behaviorally equivalent construction, representational equivalent, or causally substitutable construction would effectively reintroduce c , the constraint may also exclude a pre-specified equivalence class $Eq(c)$. Existing elements may still be reweighted, retrained, reordered, reconnected, recombined, or otherwise optimized inside the old class.

$$A_0(c; M_0) = \{ x \text{ consistent with the pre-}c \text{ availability boundary encoded by } M_0 : c \text{ is unavailable and } Eq(c) \text{ is excluded when required } \}.$$

This definition deliberately avoids treating the literal Parent P as the null model. P is one historical point; A_0 is the old system's available capability class under renewed optimization. M_0 also limits post-hoc boundary narrowing: if multiple reasonable interpretations of the old class exist, the protocol can use the broader interpretation, compare multiple A_0 variants, or return an insufficient/pending result rather than certify.

2.3 Counterfactual comparison information and refutation sufficiency

Let E be an evaluation set informationally isolated from both child generation and A_0 reoptimization. Reoptimizing, searching, enumerating, or bounding A_0 yields counterfactual comparison information B_0 . B_0 need not be a single materialized "Shadow Ancestor"; it may be the best observed A_0 configuration, a performance distribution, a confidence bound, a surrogate or algorithmic upper bound, or a formal optimum certificate. In parallel, Q_0 denotes a pre-specified refutation-sufficiency condition indicating whether the old class received an adequate falsification attempt. Q_0 may depend on matched or greater compute than child generation, multiple independent initializations or optimizers, convergence, improvement saturation, search coverage, an upper bound, or a formal completion proof.

$$B_0 = \text{CounterfactualBound}(\text{Reoptimize}(A_0(c; M_0))); \quad Q_0 = \text{RefutationSufficiency}(A_0 \text{ evidence, declared completion rule}).$$

2.4 Relative structural non-reducibility

ECSH certifies c only when the M_0 -to- A_0 boundary is consistent, Q_0 is satisfied, and C satisfies a pre-specified decision rule against B_0 on E . For a scalar performance metric in which higher is better, a simple conceptual form is:

$$\text{CERT}(c) = 1 \text{ only if } \text{BoundaryOK}(M_0, A_0) \wedge Q_0 \wedge \text{Test}(S(C; E), B_0(E)) = \text{PASS}.$$

The test can be statistical or formal, but the protocol uses A_0 rather than P as its counterfactual reference class, and the fail-closed conditions explicitly separate "old-class search was insufficient" from "old class was adequately challenged and failed." The certificate means that the observed child advantage was not reproduced by the specified pre- c capability class under the stated boundary, refutation, isolation, and decision protocol. It is therefore a conditional structural attribution, not an ontological statement about all possible programs.

3. Related Work and the Residual Problem

DGM iteratively modifies agent code and empirically validates descendants on coding benchmarks while maintaining an archive of diverse agents [1]. It demonstrates practical recursive self-improvement, but its evaluation relation is not designed to distinguish a genuinely expanded structural capability class from a better configuration inside the old class.

SGM replaces unattainable formal proof requirements with statistical confidence tests for recursive edits and introduces global risk allocation across repeated modification decisions [2]. Its central comparison is a candidate modification against the current incumbent. ECSH can use statistical tools of this general kind as an auxiliary layer, but its structural question is different: the candidate is compared with a reoptimized, c -disabled ancestral capability class, and the result controls whether c becomes available to future candidate generation.

ROAR removes features deemed important and retraining the model to benchmark feature-importance methods [3]. The methodological lesson that removal should be followed by retraining is directly relevant. ECSH extends the counterfactual target from feature removal in a fixed modeling setting to a constrained pre- c generative capability class, and couples the resulting attribution decision to the future structural search space.

GrowLibm studies which numerical primitives would be useful to add to a mathematical library and ranks mined primitives by counterfactual utility [4]. This is close in spirit because it reasons about the value of new primitives. Its objective, however, is utility-driven library learning rather than a conditional non-reducibility certificate against a candidate-disabled old capability class subjected to a declared refutation-sufficiency test before downstream structural availability is granted.

Beyond Fixed Representations distinguishes transformations inside a fixed representational frame from transformations that alter the frame itself, and identifies vocabulary and verifier gaps in open-ended AI [5]. This provides a conceptual motivation for structural verification. In the framework as described, we did not identify the ECSH combination of candidate-disabled old-class reoptimization, informationally isolated comparison, separate operational and structural decisions, and conditional update of the future generatable structure state.

PACE [6] recasts repeated self-evolution commits as paired anytime-valid sequential tests, controlling false commits under optional stopping. SEA [7] similarly places anytime-valid certificates around self-modification in a versioned harness. These works strengthen the statistical acceptor and are directly relevant to ECSH’s auxiliary repeated-test layer. Their described acceptance questions concern whether a candidate modification should be committed relative to an incumbent or fixed baseline; we did not identify a requirement that candidate-disabled reconstruction and refutation of the pre-c structural capability class condition downstream structural availability.

Mendel Gödel Machine (MGM) [8] uses comparative evidence across tasks and lineages to improve how self-modifying coding agents generate inherited edits, including reaction-norm mutation and cross-lineage hybridization. This makes inheritance explicit and therefore creates a close terminological neighbor to ECSH. MGM controls the evidence used to generate modifications; ECSH addresses a different question after a candidate exists: whether the candidate structure survives a fail-closed counterfactual challenge from the M0-constrained, c-disabled ancestral class before that structure becomes downstream-generatable.

Hyperagents [9] makes the meta-level modification procedure itself editable: a task agent and a meta agent are integrated into one editable program, allowing the mechanism that generates future improvements to change over time. This materially narrows any broad claim that meta-level self-modification or editable improvement machinery is itself distinctive. ECSH does not rely on that claim; its residual question is whether a fixed candidate structure remains non-reducible after the candidate-disabled pre-c capability class is given a declared adequate opportunity to refute the extension hypothesis, and whether that result controls downstream structural availability.

HarnessX [10] treats the agent harness—prompts, tools, memory, control flow, and related runtime interfaces—as a composable and evolvable object, with trace-driven adaptation and gating of candidate harness edits. It is therefore close prior work for claims about evolving runtime structure. ECSH addresses a different acceptance target: not whether a harness edit improves performance or avoids regression, but whether the improvement cannot be recovered by legitimate reoptimization inside the M0-constrained, c-disabled ancestral capability class before c is admitted to the future structural vocabulary.

Falsifiable Release Gates [11] requires new capabilities in a self-improving runtime to pass pre-declared, machine-checkable acceptance suites while standing invariants are preserved. This is important adjacent work for fail-closed certification and release gating. Its described gate is principally a safety/invariant release test; ECSH instead constructs a candidate-disabled ancestral counterfactual and asks a relative structural-attribution question. Passing an ECSH certificate is not a deployment-safety certificate, and passing a release-safety gate does not by itself establish ECSH-style structural non-reducibility.

Hierarchical Self-Improvement (HSI) [12] evolves a task-specific harness, an evolver that rewrites that harness, and a meta-evolver that rewrites the evolver strategy under a frozen outer anchor. HSI therefore provides especially close evidence that harnesses and improvement procedures themselves are active self-modification surfaces. Its reported objective is task-specific harness improvement under feedback and backbone bounds; in its described protocol, we did not identify candidate-disabled reoptimization or upper-bounding of a declared pre-c capability class, followed by informationally isolated structural attribution and conditional hereditary availability, as the criterion for accepting a structural birth.

Bootstrap Theory of Representational Emergence (TBER) [13] treats persistent explanatory insufficiency as a signal for re-representation. Versions publicly available before the 19 August 2026 ECSH filing already developed

explanatory-insufficiency-driven representational transition as a conceptual program; a later v4 dated 20 August 2026 adds a more developed distinction among local-corrective, representationally resolutive, and structurally recurrent resolution regimes, with regime-sensitive diagnosis and closure assessment. TBER therefore materially narrows any broad claim that residual or explanatory failure motivating representational change is distinctive. ECSH does not rely on that broad claim: its narrower object is a fixed candidate structure *c*, a versioned pre-*c* boundary *M0*, a *c*-disabled ancestral class *A0* that is reoptimized/searched/enumerated/upper-bounded under *Q0*, an informationally isolated comparison against *B0*, and a conditional update of downstream structural availability only after the declared relative non-reducibility test passes. The post-filing v4 material is included here as later technical context, not as evidence of pre-filing disclosure.

SkillMaster [14] integrates skill-bank editing into the agent policy and evaluates each proposed skill edit by counterfactual utility on related probe tasks, comparing rollouts under the original and temporarily modified skill banks. This is important prior work against any broad claim that counterfactual evaluation of reusable capability edits is new. Its reference is the current skill bank rather than a reconstructed, candidate-disabled pre-*c* capability class that is independently reoptimized under a declared refutation-sufficiency rule, and its counterfactual signal trains skill-editing decisions rather than certifying relative structural non-reducibility before downstream structural availability changes.

Phantom Guardrails [15] demonstrates that automated harness optimizers can fabricate nonexistent failure classes and accumulate unnecessary guardrails in add-only improvement loops. This provides direct evidence that persistent structural additions can be wrongly justified even when the measured target is already satisfied. Its oracle asks whether an asserted failure actually occurred; it does not reconstruct *A0*, give the candidate-disabled old class a renewed optimization opportunity under *Q0*, or certify that a candidate advantage lies outside that old class.

Self-Evolving Agent Harnesses via Gated Semantic Quality-Diversity [16] explicitly separates semantic patch proposal from deterministic sampling, measurement, and significance-based credit assignment, with sealed evaluation used to assess generalization. This materially narrows any broad claim that proposer/credit separation, deterministic gating, or sealed-test evaluation is distinctive. Its acceptance target is trustworthy performance credit for a harness patch; it does not use an *M0*-constrained, candidate-disabled ancestral class as the counterfactual and does not condition future structural availability on surviving old-class reoptimization under *Q0*.

Auditing Harness Tampering in Self-Improving Agents [17] studies self-modifications that compromise integrity obligations such as authorization, provenance, and completeness, and reports that such edits can persist in lineages. This is especially relevant to ECSH's boundary-integrity requirement and independently strengthens the case for auditable provenance around structural changes. Its target, however, is tampering classification and localization rather than relative structural attribution against an *M0*-constrained, *c*-disabled ancestral capability class under a refutation-sufficiency condition. The work first appeared after the 19 August 2026 ECSH filing and is included as later technical context.

Within the references and primary sources reviewed for this manuscript through 17 September 2026, including work on counterfactual skill-utility evaluation, phantom structural additions, deterministic harness crediting, and harness-tampering audits, we did not identify a single work that expressly combines the following as one control protocol: versioned pre-*c* capability-state reconstruction; candidate-disabled old-class reoptimization, search, enumeration, or upper-bounding with an explicit refutation-sufficiency check; informationally isolated structural attribution distinct from operational adoption; and fail-closed conditional update of future structural availability. For chronology, only material publicly available by 19 August 2026 is treated as potentially pre-filing technical context; later versions and later publications are discussed only to position the research. This is a limited research-literature statement, not a claim of exhaustive searching and not a conclusion about legal novelty, inventive step, patent validity, infringement, or freedom to operate; combinations of adjacent prior art remain relevant.

Table 1. Relationship to adjacent methods (high-level functional comparison).

Method	Primary comparison target	Reoptimizes <i>c</i> -disabled pre- <i>c</i> class?	Structural-attribution decision?	Controls future structural availability?
DGM [1]	Descendant / archive quality	No	Not its target	Archive growth; no ECSH-style gate
SGM [2]	Candidate vs incumbent	No	Modification acceptance	Candidate adoption under risk control

Method	Primary comparison target	Reoptimizes c-disabled pre-c class?	Structural-attribution decision?	Controls future structural availability?
ROAR [3]	Retrained model after removal	Partial analogue	Feature-importance validation	No
GrowLibm [4]	Utility of candidate primitives	No ECSH-style old-class defense	Utility ranking	Library expansion by utility
Beyond Fixed Representations [5]	Vocabulary / verifier gap	Conceptual only	Motivates verifier need	Conceptual only
PACE [6]	Candidate vs incumbent (anytime-valid)	No	Modification acceptance	Candidate commit; not structural gate
SEA [7]	Modification vs versioned baseline	No	Modification certificate	Admits tested modification
MGM [8]	Comparative task / lineage evidence	No ECSH-style old-class defense	Edit-generation / inheritance control	Archive-based inherited edits
ECSH	Child vs M0-constrained old class	Yes, with boundary + Q0	Yes, relative to M0/A0/Q0	Yes; fail-closed after certification
Hyperagents [9]	Task/meta-agent performance; editable meta-improvement	No ECSH-style old-class defense	Self-improvement mechanism	Editable improvement machinery
HarnessX [10]	Candidate harness quality / regression checks	No ECSH-style old-class defense	Harness adaptation / gating	Commits improved harness edits
Falsifiable Release Gates [11]	Capability vs acceptance suite / invariants	No	Safety / release certification	Release gated on invariant checks
HSI [12]	Task-specific harness performance across hierarchical scopes	No ECSH-style old-class defense	Harness/evolver/meta-evolver improvement	Evolved harness strategy; not ECSH heredity gate
SkillMaster [14]	Original vs modified skill bank on related probes	No; compares current bank vs candidate edit	Counterfactual utility / edit credit	Updates reusable skill bank through learned edit decisions
Phantom Guardrails [15]	Asserted failure vs deterministic oracle	No	Detects fabricated failure justification	Studies persistent add-only guardrails; no ECSH-style heredity certificate
Gated Semantic QD [16]	Candidate harness patch vs baseline under deterministic crediting	No	Performance credit / significance gate	Promotes credited harness patches
Harness Tampering Audit [17]	Harness edit vs integrity obligations	No	Tampering classification / localization	Audits lineage persistence; not a structural-birth gate

4. ECSH Protocol

4.1 Objects and system state

ECSH assumes a system that can generate candidate descendants and whose future candidate generator has an explicit or implicit structural state H . H may be a registry of operators, a primitive store, a grammar, a tool permission set, a search mask, enabled branches, selection weights, generation probabilities, or equivalent program state. The protocol also requires $M0$, an auditable representation of pre- c availability sufficient to reconstruct or constrain $A0$, and $Q0$, a declared criterion for deciding whether the $A0$ refutation attempt was sufficiently strong. The protocol is agnostic to whether the proposal originates from the system itself, an external trainer, an evolutionary controller, program synthesis, AutoML, a compiler, or another agent.

Table 2. Core notation.

Symbol	Meaning
P	Parent or pre-modification configuration.
C	Fixed child configuration under structural certification.
c	Candidate structural element or interdependent composite candidate.
$M0$	Ancestral capability-state information identifying the pre- c availability boundary.
$A0(c; M0)$	Ancestral capability class with c unavailable and generation/reoptimization restricted by $M0$.
$B0$	Counterfactual comparison information derived by reoptimizing, searching, or bounding $A0$.
$Q0$	Pre-specified refutation-sufficiency condition for the $A0$ falsification attempt.
E	Evaluation data informationally isolated from child generation and $A0$ reoptimization.
H	Downstream generatable structural-space state.
$CERT(c)$	Conditional structural non-reducibility result relative to $M0$, $A0$, $Q0$, E , and the decision rule.

4.2 Step 1: identify and hold the child identity

After a candidate child C is produced, the structural delta is parsed to identify c . During certification, the identity of C must remain stable. This can be implemented by an immutable snapshot, content hash, model-checkpoint identifier, container digest, version identifier, or an equivalent mechanism. Physical copying is not essential; reproducible identity is.

4.3 Step 2: reconstruct and validate the ancestral capability class

The old capability class is reconstructed or constrained from $M0$: structural vocabulary, generation or mutation rules, permissions, dependencies, type constraints, masks, versions, or equivalent pre- c state. Direct use of c is forbidden, and predefined equivalence restrictions may block renamed, wrapped, behaviorally equivalent, representationally equivalent, or causally substitutable reintroductions. A boundary-integrity check verifies that $A0$ does not exclude capabilities genuinely available before c and does not include structures that became available only after c . When the boundary remains materially ambiguous, ECSH can compare broader or multiple $A0$ variants or return INSUFFICIENT/PENDING rather than certify.

4.4 Step 3: reoptimize or upper-bound A0 and establish Q0

Reoptimization proceeds under a pre-specified refutation-sufficiency rule Q0. Examples include spending at least or a declared multiple of the computational budget used to obtain C, satisfying convergence across independent initializations or multiple optimizers, observing sufficiently small improvements for a specified run length, reaching declared search coverage, exhausting an enumerable space, or deriving an upper bound or formal completion certificate. The reoptimizer need not be the same algorithm that generated C; optimizer portfolios are permitted as long as the M0/A0 constraint is maintained. If Q0 is not met, certification fails closed rather than interpreting search failure as structural irreducibility.

The point is adversarial: ECSH attempts to falsify the structural-birth hypothesis by finding the strongest c-free explanation legitimately available inside the old class. Boundary error and search insufficiency are therefore first-class failure modes rather than silent sources of positive certification.

4.5 Step 4: evaluate on informationally isolated data

E must not be used to generate C or to reoptimize A0. It can be a fresh held-out set chosen after C is fixed, or a sealed set created earlier whose contents and results remain hidden from both optimization processes. Isolation can be maintained by process boundaries, access control, third-party evaluation, commitment schemes, or equivalent mechanisms.

4.6 Step 5: certify structural non-reducibility

After BoundaryOK(M0, A0) and Q0 are satisfied, compare C with B0 on E using a pre-specified decision rule. The performance variable may be task success, loss, reward, safety-constraint satisfaction, latency, resource efficiency, robustness, or a multi-objective criterion. Statistical tests, confidence bounds, e-values, bootstrap procedures, Bayesian rules, distributional dominance, or formal bounds can be used. These statistical choices are auxiliary: the core structural distinction is the M0-constrained counterfactual reference class and the fail-closed sufficiency check.

4.7 Step 6: update downstream generatability only on certification

If BoundaryOK(M0, A0), Q0, and CERT(c)=PASS all hold, H is changed so that c or a rule enabling c becomes available to future candidate generation. The implementation may add c to a registry, enable a tool or operator, release a search mask or branch, grant a permission, or move a generation weight/probability from an effectively unavailable state to an available state. If any required condition is not satisfied, the structural state remains closed to c; the result may be FAIL, INSUFFICIENT, or PENDING without blocking separate operational use of C.

$H(t+1) = \text{Enable}(H(t), c)$ only if $\text{BoundaryOK}(M0, A0) \wedge Q0 \wedge \text{CERT}(c)=\text{PASS}$; otherwise $H(t+1)=H(t)$ with respect to c.

4.8 Operational adoption is a separate channel

ECSH does not require rejecting a useful child merely because c is not structurally certified. A child that improves latency, robustness, or task performance may be retained operationally while c remains unavailable to future mutation or structural generation. Conversely, a structurally attributable capability can still be blocked from deployment by an independent safety or product policy. This separation prevents an ordinary optimization win from automatically rewriting the system's future search vocabulary.

5. Reference Algorithm

Algorithm 1 gives implementation-neutral pseudocode for the narrow ECSH core.

```

Algorithm 1: ECSH structural-attribution gate
Input: fixed child C; candidate structure c; ancestral capability-state information M0; pre-
c structural state H0;
refutation-sufficiency rule Q0-rule; isolated evaluation protocol E; decision rule T
1. Preserve the identity of C for the duration of certification.
2. Construct A0 from M0 while disabling c and required equivalents.
3. Verify boundary integrity between M0 and A0; if unresolved, return INSUFFICIENT/PENDING
and keep c unavailable.
4. Reoptimize, search, enumerate, or upper-bound A0 under the declared Q0-rule.
5. If Q0 is not satisfied, return INSUFFICIENT and keep c unavailable.

```

```

6. Derive counterfactual comparison information B0 from the resulting A0 evidence.
7. Evaluate C and B0 using data E isolated from both candidate generation and the A0 search.
8. z <- T(C, B0, E).
9. if z = PASS:
  H1 <- make c (or an enabling rule for c) available to downstream generation.
else: H1 <- H0 with c remaining unavailable.
10. Separately decide whether C is operationally retained or deployed.
Output: structural result z; boundary/Q0 evidence; updated structural state H1; separate
operational decision.

```

Optional corroboration. Ablating c from C and checking that the advantage disappears, or requiring replication across independent task families/environments, can strengthen attribution. These checks supplement rather than replace the M0-constrained reoptimized ancestral counterfactual, boundary-integrity check, and refutation-sufficiency condition Q0.

6. Synthetic Control

6.1 Purpose

The experiment is a calibration control in which the true candidate class and pre-c availability boundary are known by construction. This lets M0/A0 boundary integrity be fixed rather than inferred from a real evolving system. It is designed to measure whether different gates mistakenly label old-class tuning or null changes as heritable structural births while retaining sensitivity to genuine structural-extension candidates. It is not a simulation of a complete real-world self-modifying agent, and it does not establish that the synthetic structural labels correspond to ontologically new capabilities outside the generator.

6.2 Candidate population and Shadow budget

Each independent lifetime contains 200 candidates: 80 tuning candidates that remain inside the existing capability class, 20 structural-extension candidates, and 100 null candidates. The ancestral comparison uses 16 old-class configurations, with 800 Shadow-reoptimization samples per configuration. This frozen search design acts as the declared synthetic Q0 rule for the reported calibration; it does not imply that the same budget is sufficient in arbitrary real systems. After the candidate is fixed, 60 fresh paired held-out observations are used for structural comparison. Candidate-class labels are used only for post-hoc scoring of recall, precision, and false births.

6.3 Compared gates

1. Child>Parent only: register whenever the child outperforms its immediate parent, without a structural counterfactual.
2. Parent-relative statistical gate: statistically certify the child against the parent/incumbent, but do not reoptimize the c-disabled old capability class.
3. Shadow comparison without cumulative error control: compare against the reoptimized old class but omit the repeated-test error-control layer used in the reproduced ECSH configuration.
4. ESH: use the M0-constrained, c-disabled ancestral reoptimization, declared Q0 sufficiency rule, isolated held-out comparison, downstream structural gate, and a pre-specified cumulative error-control schedule for repeated structural tests. The error-control schedule is an auxiliary statistical component rather than the claimed source of structural attribution.

6.4 Metrics

- Structural-extension recall: fraction of known structural-extension candidates that pass the structural gate.
- False heritable births per lifetime: number of tuning or null candidates incorrectly made available as structural births, averaged over lifetimes.
- Precision: fraction of registered candidates that are known structural-extension candidates in the synthetic generator.
- Lifetime false-birth incidence: fraction of lifetimes containing at least one false structural birth; reported for the canonical run.

7. Results

7.1 Canonical run and seed robustness

The canonical run used seed 20260816 for 1,200 lifetimes on a local Windows execution environment. It produced 97.55% structural-extension recall, 24 false heritable births, 0.020 false births per lifetime, 2.0% of lifetimes containing at least one false birth, and 99.8976% precision. After the patent submission, the same frozen synthetic configuration was run on fresh seeds 20260819, 20260820, and 20260821. Each fresh seed contained 1,200 lifetimes (240,000 candidate rows), giving a fresh three-seed mean recall of 97.5014%, mean false births per lifetime of 0.01944, and mean precision of 99.9004%. For the public-evidence rebuild on 17 September 2026, the exact frozen public reproduction source and specification were used to regenerate the anchor and all three fresh seeds from scratch; all four independent CSV recomputations matched the reported aggregate values. Across the four seeds, 4,800 lifetimes and 960,000 candidate rows were evaluated.

Table 3. Reproduced synthetic-control results across seeds.

Seed	Lifetimes	Structural recall (%)	False births / lifetime	Precision (%)
20260816 (anchor)	1200	97.5500	0.02000	99.8976
20260819 (fresh)	1200	97.5625	0.01833	99.9061
20260820 (fresh)	1200	97.4208	0.01667	99.9145
20260821 (fresh)	1200	97.5208	0.02333	99.8805
Fresh mean (19–21)	—	97.5014	0.01944	99.9004
All four seeds	—	97.5135	0.01958	99.8997

7.2 Counterfactual target matters more than Parent-relative acceptance

On canonical seed 20260816, the naïve Child>Parent rule produced 126.1117 false heritable births per lifetime. Adding a Parent-relative statistical gate reduced this to 77.6092, but still left a large attribution error because the reference remained the historical parent. Replacing the parent with a reoptimized c-disabled Shadow comparison reduced false births to 5.4383 per lifetime even without cumulative error control. The reproduced ECSH configuration reduced the rate further to 0.020 per lifetime.

Table 4. False heritable births per lifetime on canonical seed 20260816.

Gate	Counterfactual reference	False births / lifetime
Child>Parent only	Immediate parent	126.1117
Parent-relative statistical gate	Immediate parent with statistical acceptance	77.6092
Shadow, no cumulative error control	Reoptimized c-disabled old class	5.4383
Reproduced ECSH	Reoptimized c-disabled old class + repeated-test control	0.020

7.3 What the synthetic result supports

The large gap between Parent-relative gates and the Shadow-based gates supports the narrower methodological claim that structural attribution is poorly approximated by incumbent comparison when the old capability class can still be reoptimized. The additional improvement from repeated-test error control shows that multiplicity can matter in long sequences of candidate certifications. The fresh-seed reruns show that the canonical gate behavior persists under new random seeds, but they do not address real-system boundary inference: in this synthetic control M0/A0 membership is known by construction and Q0 is defined by the frozen search budget. The experiment therefore does not show that ECSH has discovered a real new capability, that the chosen synthetic distributions model real self-modifying systems, or that M0, A0, and Q0 can always be specified correctly in practice.

8. Failure Modes and Limitations

8.1 The certificate is conditional on M0 and the A0 boundary

M0 and A0 jointly define the counterfactual reference class. If M0 is incomplete or interpreted too narrowly, the old system may be prevented from using capabilities it genuinely possessed and false structural births become easier. If

the boundary is too broad or permits c-equivalent leakage, the counterfactual can improperly use the innovation being tested and suppress a true structural extension. Auditable versioning, manifests, hashes, permissions, dependencies, or equivalent pre-c evidence are therefore central. When boundary ambiguity cannot be resolved, fail-closed non-certification is preferable to silently selecting a favorable A0.

8.2 Search insufficiency can mimic irreducibility

For large capability classes, the exact optimum may be intractable. A weak Shadow search can fail to find an old-class solution that actually exists. ECSH therefore requires a declared refutation-sufficiency condition Q0, using sufficient budget relative to child generation, multiple initial conditions or optimizer portfolios when appropriate, convergence or saturation criteria, search coverage, or upper-bound methods when available. Failure to meet Q0 yields INSUFFICIENT/PENDING rather than a positive structural certificate. Even when Q0 is met, a certificate remains relative to the stated search procedure unless a formal upper bound or completion proof is obtained.

8.3 Evaluation isolation can fail

If held-out information leaks into candidate generation or A0 reoptimization, both Child and Shadow estimates can be biased. Sealed evaluation must therefore be treated as part of the structural-certification boundary, not merely as good benchmark hygiene.

8.4 Candidate boundaries can be composite

A capability may arise only from an interaction among several changes. Testing each delta independently can miss such synergy. ECSH therefore permits a mutually dependent set of changes to be treated as one composite candidate. This increases combinatorial cost and makes candidate decomposition itself an open problem.

8.5 Certification can be expensive

The protocol intentionally spends computation to give the old capability class a strong defense. This is suitable when structural-space expansion is relatively rare and consequential, but may be too costly as a gate for every ordinary parameter update. In practice, systems may reserve ECSH for changes that alter structural vocabulary, permissions, tools, representations, or generation rules.

8.6 Statistical control is not the structural novelty claim

Repeated testing creates its own error-control problem. The reproduced synthetic configuration uses a pre-specified cumulative error-control schedule, but this layer is conceptually separable from the structural attribution mechanism and overlaps with a mature statistical literature, including SGM, PACE, and SEA. The central methodological move examined here is not a new repeated-testing method: the protocol changes the reference from the historical incumbent to an M0-constrained, candidate-disabled ancestral capability class, requires an explicit refutation-sufficiency check Q0, and couples the conditional structural result to future structural availability.

8.7 Synthetic external validity

The current evidence is deliberately limited. Because the true candidate class and pre-c boundary are known by construction, the synthetic control can measure false structural registrations precisely and does not test the hardest real-world problem: reconstructing M0, defining A0 without hindsight, and determining an adequate Q0 for an open-ended system. Real self-modifying systems do not provide such ground truth. The next empirical question is whether ECSH can discriminate genuine relative structural extensions from strong old-class reoptimization on real tasks without making A0 trivially weak, prohibitively broad, or Q0 economically impractical.

8.8 ECSH is not a deployment-safety certificate

A structural capability can be dangerous, brittle, misaligned, or economically useless. ECSH answers a narrower attribution question. Deployment, safety, policy, and product acceptance require separate gates.

9. Implications for Open-Ended Self-Modification

Open-ended systems need a mechanism for changing not only values inside a fixed search space but also the search space itself. That creates a verifier asymmetry: ordinary optimization can be judged against a fixed objective, while

structural innovation changes what later searches are able to express. If every useful descendant automatically expands the future structural vocabulary, tuning artifacts and noisy improvements can accumulate as if they were new primitives. If structural expansion is never allowed, the system remains trapped in a fixed representational frame.

ECSH provides one operational compromise. It allows aggressive ordinary optimization while making structural-space expansion adversarial and auditable: M0 records what the old system was allowed to use, A0 reconstructs that old capability class without c, and Q0 requires the old class to receive a declared adequate falsification attempt. Only candidates that survive that counterfactual defense are allowed to modify downstream generatability. This shifts “innovation” from a label attached to a good descendant into a controlled, evidence-bearing state transition in the generator itself.

The approach is relevant wherever later candidate generation can inherit newly enabled operators, tools, representations, model modules, workflow steps, compiler rewrites, database strategies, or program-synthesis productions. The same logic can be implemented in a single lineage, a population, a distributed controller, or an external orchestration system, provided the pre-c capability boundary and downstream availability state are explicit enough to audit.

10. Reproducibility and Release Plan

The reported numbers come from an independently reconstructed ECSH-Core synthetic control rather than from an earlier legacy generator. The public-evidence rebuild on 17 September 2026 used the frozen public source archive `NAGI_ECSH_REPRO_V1_0_0.zip` (SHA-256 `ac8d1f0765561bfc5b67442e58636a601602073049b24608df4285a3080912e4`), containing `ecsh_repro.py` (SHA-256 `73e425120b6ca647e46bd660affabfc8bcf3a53a4e947d4dcc8a264ddb8fb96`) and `ECSH_REPRO_SPEC.json` (SHA-256 `bbfb8f6d7a065365f411be20dff70972fdaa30ab4cd3a3686cbb16f94217ecfc`). The same four reported seeds—20260816, 20260819, 20260820, and 20260821—were freshly regenerated, producing 240,000 candidate rows per seed and 960,000 rows total, and an independent CSV-only recomputation reproduced all manuscript aggregates. A bounded local search did not recover the earlier candidate-level historical raw artifact. Accordingly, the public release must describe the 17 September CSVs as fresh deterministic regeneration from the frozen source/specification and must not represent them as recovered historical raw data.

The public manuscript and repository are restricted to configurations supported by the submitted patent disclosure. The fresh seeds and the 17 September evidence rebuild reuse the disclosed synthetic configuration and are presented as post-submission verification and provenance hardening, not as a newly introduced technical mechanism. The paper is identified by DOI 10.5281/zenodo.22806440, and the accompanying software/evidence release is identified by DOI 10.5281/zenodo.22806442. The public repository is <https://github.com/yonezaemon1-hub/ecsh-structural-capability-birth-certification>. The software/evidence release contains the frozen source archive, fixed specification, freshly regenerated candidate-level raw evidence, independent-verification JSON, evidence report, and SHA-256 manifest. Any future technical configuration that materially extends the disclosed mechanism should be handled separately before public release.

11. Conclusion

Improvement relative to a parent is not enough to establish structural capability expansion. ECSH turns structural attribution into a fail-closed adversarial counterfactual test: preserve the child, reconstruct the pre-c availability boundary from M0, disable the candidate structure, challenge the resulting A0 under a declared refutation-sufficiency condition Q0, compare against B0 on informationally isolated data, and update the future generatable structure state only when boundary integrity, refutation sufficiency, and relative non-reducibility all pass. Operational adoption remains a separate decision.

In a synthetic control with known candidate classes, Parent-relative rules produced large numbers of false heritable births, while reoptimized ancestral comparison dramatically reduced them. The reproduced ECSH configuration achieved 97.55% recall and 0.020 false births per lifetime on canonical seed 20260816. Three fresh post-submission seeds produced mean recall 97.5014%, mean false births per lifetime 0.01944, and mean precision 99.9004%. These results are evidence about gate calibration and seed robustness in a synthetic setting, not a proof of real-world

structural birth. The central empirical challenge is now to test whether the same distinction remains useful when M0, A0, candidate boundaries, Q0, and evaluation isolation must be established in real self-modifying systems.

12. Scope and Research-Claim Boundary

For clarity, this manuscript does not claim any of the following:

- That the synthetic control proves emergence of a new real-world capability.
- That satisfying a finite Q0 search rule proves absolute impossibility inside all old implementations; finite-search certificates remain conditional on M0, A0, Q0, and the stated protocol.
- That ECSH subsumes statistical modification-acceptance methods such as SGM, PACE, or SEA; repeated-test statistics are treated as an auxiliary layer.
- That simple ablation alone is sufficient for structural attribution.
- That a structurally certified capability is safe to deploy.
- That the present literature search establishes legal novelty, inventive step, patent validity, or infringement.
- Any broad claim to inventing self-modification, editable meta-level improvement, evolvable agent harnesses, machine-checkable release gating, counterfactual utility evaluation of reusable skill edits, proposer/credit separation, deterministic harness crediting, harness-tampering auditing, residual-driven re-representation, or explanatory-insufficiency-triggered representational transition; these are treated as adjacent prior art rather than ECSH contributions.
- Any claims regarding negative heredity, capability-death certificates, hereditary revocation, or related mechanisms outside the narrow ECSH core.

References

- [1] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. “Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents.” arXiv:2505.22954, 2025. [arXiv](#)
- [2] Xuening Wu, Shenqin Yin, Yanlan Kang, Xinhang Zhang, Qianya Xu, Zeping Chen, and Wenqiang Zhang. “SGM: A Statistical Gödel Machine for Risk-Controlled Recursive Self-Modification.” arXiv:2510.10232, 2025. [arXiv](#)
- [3] Sara Hooker, Dumitru Erhan, Pieter-Jan Kindermans, and Been Kim. “A Benchmark for Interpretability Methods in Deep Neural Networks.” Advances in Neural Information Processing Systems 32, 2019; arXiv:1806.10758. [arXiv](#)
- [4] Jonas Regehr, Mitch Briles, Zachary Tatlock, and Pavel Panchekha. “Numerical Superoptimization for Library Learning.” arXiv:2603.24812, 2026. [arXiv](#)
- [5] Yuan Cao and Haiqian Yang. “Beyond Fixed Representations: The Vocabulary and Verifier Gaps in Open-Ended AI.” arXiv:2607.09560, 2026. [arXiv](#)
- [6] Zayx Shawn. “PACE: Anytime-Valid Acceptance Tests for Self-Evolving Agents.” arXiv:2606.08106, 2026.
- [7] Biswa Sengupta. “Self-Evolving Agents with Anytime-Valid Certificates.” arXiv:2607.00871, 2026.
- [8] Changzhi Liu, Yilun Liu, Sikuan Yan, Volker Tresp, and Yunpu Ma. “Mendel Gödel Machine: Recursive Self-Improving Coding Agents via Comparative Evolution.” arXiv:2608.07645, 2026.
- [9] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. “Hyperagents.” arXiv:2603.19461, 2026.
- [10] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. “HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry.” arXiv:2606.14249, 2026.
- [11] Deepak Soni. “Falsifiable Release Gates for Self-Improving Systems.” arXiv:2607.13070, 2026.
- [12] Tailin Zhou. “Hierarchical Self-Improvement: A Framework for Task-Specific Evolvable Agent Harnesses.” arXiv:2608.08466, 2026.
- [13] Jacques Raynal, Pierre Slangen, Elsa Raynal, and Jacques Margerit. “Bootstrap Theory of Representational Emergence (TBER): Explanatory Insufficiency, Transition Regimes, and the Emergence of New Representational Levels.” arXiv:2606.07303, 2026. v4, 20 August 2026.
- [14] Min Yang, Jinghua Piao, Xu Xia, Xiaochong Lan, Jiaju Chen, Yongshun Gong, and Yong Li. “SkillMaster: Toward Autonomous Skill Mastery in LLM Agents.” arXiv:2605.08693, 2026.
- [15] Su Wang, Pin Qian, Yifan Lin, Jingzhou Xu, Yihang Chen, Xiaochong Jiang, Lifei Liu, and Haoran Yu. “Phantom Guardrails: When Self-Improving Agent Harnesses Fix Failures That Never Happened.” arXiv:2607.13083, 2026.
- [16] Xiaotian Luo, Fengxingyu Wang, Chuanrui Hu, Dizhan Xue, and Yafeng Deng. “Self-Evolving Agent Harnesses via Gated Semantic Quality-Diversity.” arXiv:2607.13683, 2026.
- [17] Xing Wang, Xiaoyi Zhang, and Jie Shao. “Auditing Harness Tampering in Self-Improving Agents.” arXiv:2609.00069, 2026.